

Invited: New Solutions on LLM Acceleration, Optimization, and Application

Yingbing Huang¹, Lily Jiaxin Wan¹, Hanchen Ye¹, Manvi Jha¹,
Jinghua Wang¹, Yuhong Li¹, Xiaofan Zhang², Deming Chen¹

{yh21, wan25, hanchen8, manvij2, jinghua3, leeyh, dchen}@illinois.edu, xiaofanz@google.com

¹University of Illinois Urbana-Champaign, ²Google

ABSTRACT

Large Language Models (LLMs) have revolutionized a wide range of applications with their strong human-like understanding and creativity. Due to the continuously growing model size and complexity, LLM training and deployment have shown significant challenges, which often results in extremely high computational and storage costs and energy consumption. In this paper, we discuss the recent advancements and research directions on (1) LLM algorithm-level acceleration, (2) LLM-hardware co-design for improved system efficiency, (3) LLM-to-accelerator compilation for customized LLM accelerators, and (4) LLM-aided design for HLS (High-Level Synthesis) functional verification. For each aspect, we present the background study, our proposed solutions, and future directions. An extended version of this work can be found at: <https://arxiv.org/abs/2406.10903>.

KEYWORDS

Large Language Models (LLMs), High-Level Synthesis (HLS), Acceleration, Functional Verification, Hardware Design

1 INTRODUCTION

In recent years, Large Language Models (LLMs) have emerged as powerful tools across various domains, revolutionizing natural language processing, information retrieval, LLM-aided design, and others. In this paper, we explore innovative solutions and directions for accelerating, optimizing, and applying LLMs. We focus on four key areas: 1) LLM algorithm-level acceleration aiming to enhance the efficiency of core algorithms driving these models. 2) LLM-hardware co-design optimizing LLM models and accelerators for improved performance and energy efficiency. 3) LLM-to-accelerator compilation, tailoring specialized accelerators for efficient LLM task execution. 4) An important case study of LLM-aided design for high-Level Synthesis (HLS) functional verification.

Each aspect presents unique challenges and opportunities, motivating the need for innovative research solutions. For each aspect, we review some existing works and present some of our new solutions in order to overcome the challenges. Additionally, we outline promising future directions. Our goal is to advance LLM technology for broader adoption and impactful utilization across various application domains.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06

<https://doi.org/10.1145/3649329.3663517>

2 LLM ALGORITHM-LEVEL ACCELERATION

2.1 Background Study

LLMs are great for coding assistance, answering questions, and summarizing content. However, their autoregressive nature makes decoding memory-intensive and hard to parallelize, affecting time-sensitive applications such as ChatBots or Copilot. This section focuses on algorithmic strategies to accelerate LLMs' inference, allowing easier application across different platforms.

2.1.1 KV Cache Optimization. KV Cache in LLMs stores previously computed attention values and keys. This caching proves particularly effective for complex tasks like chain-of-thought [26] reasoning or information retrieval [17]. However, it introduces overheads like setup time, extra memory for cache storage, and the complexity of managing cache validity when the sequence length or batch size increases. Several strategies have been developed to enhance KV Cache efficiency. One key approach is through advanced KV Cache management techniques. For instance, the VLLM [15] introduces PagedAttention which stores keys and values in segmented memory blocks, allowing for more efficient retrieval during attention calculations. Additionally, solutions like Hydragen [12] employ a Shared-prefix KV Cache strategy, greatly improving cache reuse rates by leveraging common sequences. Another significant advancement is the use of KV Cache compression [36], which implements eviction policies to selectively retain tokens in the cache, guided by a scoring function based on cumulative attention.

2.1.2 Parallel Decoding. Parallel decoding presents a unique approach by executing multiple decoding steps simultaneously to reduce the overall steps needed, diverging from traditional methods. It typically involves a smaller "draft" model predicting several upcoming words, which are then collectively assessed by the main LLM. This technique is specifically adapted for LLM efficiency. Recent advancements in parallel decoding for LLMs include techniques by Leviathan et al. [16] and Chen et al. [2], which introduce a resampling strategy to align small and large model predictions with the LLM's distribution, ensuring output consistency. Stern et al. [21] explore multi-token predictions from a single forward pass using a linear projection layer and a tree-based decoding structure to improve decoded sequence acceptance. Additionally, Santilli et al. [20] and Fu et al. [7] adapt Jacobi and Gaussian-Seidel algorithms [19] for parallelizing decoding, incorporating n-gram reuse and attention masks to enhance LLM efficiency.

2.2 Proposed Works

In our proposed work [1], we revisit the concept of parallel decoding with a new perspective, noting that current research primarily aims to boost generation speed through draft models. Yet, hosting dual-sized models on a server presents challenges. Moreover,

applications necessitate flawless quality and comprehension of instructions. To tackle this, we present a novel approach using multiple decoding heads as the adapter for prediction, coupled with an optimized tree-based decoding strategy, enhancing efficiency in scenarios where this is vital. Additionally, the proposed method can address the challenges associated with speculative decoding, where for each model, obtaining a small model either from scratch or from distillation is non-trivial. Our experiments demonstrate that limited-resource fine-tuning can achieve over $2.2\times$ speedup without compromising generation quality, while full fine-tuning further improves the speedup to $2.3\text{-}2.8\times$ on models with various sizes. Furthermore, parallel decoding amplifies computational load due to increased matrix operations for multi-token validation per step. By employing an optimized tree-based attention mechanism, we strive to minimize the overhead introduced by parallel decoding. Our focus on optimization of both fine-tuning and inference with the decoding adapter in the context of speculative decoding presents a novel direction for enhancing LLM performance.

2.3 Future Directions

Tailored Algorithmic Optimization. We advocate for a model-specific optimization approach, acknowledging the complexity and diversity of LLMs, especially in parallel decoding for large models. The goal is to develop predictive frameworks for selecting optimal strategies based on model size, task complexity, and performance goals, aiming to boost efficiency, adaptability, and overall model performance.

Advancements in KV Compression and Speculative Decoding. Leveraging KV compression, we see opportunities for notable improvements in tasks with large input prompts, like summarization and multi-round chats, where precise prompt compression will be crucial for maintaining retrieval accuracy and understanding. We also aim to tackle the computational load from speculative decoding, which evaluates multiple tokens at each step, by optimizing to cut down on the associated matrix multiplication overhead. These refinements are intended to boost LLM efficiency and effectiveness in practical uses while maintaining high accuracy.

3 LLM-HARDWARE CO-DESIGN

3.1 Background Study

The exceptional capabilities of LLMs are countered by their significant memory and computational overheads. Addressing these, LLM-hardware co-design, inspired by the DNN/Accelerator co-design methodology [8] [9], customizes hardware to meet these demands and provides insights to optimize LLM architectures. Specialized accelerators like GPUs, TPUs, and FPGAs enhance parallel processing and memory capacity, aiding efficient LLM execution. At the same time, software strategies such as model distillation, pruning, and quantization reduce LLM size and complexity, making them more adaptable to hardware constraints.

Customized LLM accelerators like Tensor Processing Units (TPUs) are designed to efficiently handle matrix operations which are fundamental for LLM attention and linear layers. Beyond ASICs, FPGA-based accelerators like DFX [10], which utilizes model parallelism and enables rapid concurrent execution of workloads, and FlightLLM [33], an adaptable sparse DSP framework, show their capabilities in handling diverse applications and scenarios.

For LLM designs, researchers have investigated hardware-aware model compression technologies to optimize LLM architecture. FlashAttention[5] reduces the number of High Bandwidth Memory (HBM) accesses by using tiling techniques in attention computations and extends it to block-sparse attention. PagedAttention [15] divides the KV cache into blocks and manages blocks as pages in OS's virtual memory, reducing the internal and external fragmentation and thus increases the efficiency within a single request. In addition, model distillation, pruning, and quantization have proven to improve hardware efficiency for LLM deployment. MLFS [14] freezes a base model and stores many small low rank adapter matrices, which maintains high quality encoder models on edge applications and reduces training time. LLM.int8 [6] develops a two-part vector-wise quantization procedure and a new mixed-precision decomposition scheme, enabling models like OPT-175B on a single server with consumer GPUs. SmoothQuant [27] uses a per-channel smoothing factor to handle outliers in activations and achieves up to $1.56\times$ speedup and $2\times$ memory reduction. ViTCoD [32] prunes attention maps to either dense or sparse patterns and designs an accelerator that coordinates between these two workloads to boost hardware utilization, while integrating on-chip encoder and decoder engines.

3.2 Proposed Works

Following the LLM-hardware co-design method, we propose AutoDistill [35] to provide a co-design framework that integrates model compression, architecture exploration, and multi-objective optimization. It can identify promising models with the consideration on hardware performance and model quality. We also propose a pruning-aware quantization strategy by combining two effective LLM compression methods, pruning and quantization [25]. We observe a similar sparsity distribution pattern of attention heads across various datasets, which could be used to smartly choose either completely pruning (0 bit) or different quantization precision (4, 8, 16 bits) for attention heads on individual layers with negligible overhead, based on the hardware level objective.

3.3 Future Directions

System-aware algorithmic optimization. While existing LLM training and serving leverage accelerators (e.g., GPUs and TPUs), they often underutilize other hardware resources. It creates opportunities to explore partitioning and offloading strategies to idle or lightly loaded devices. For example, host offloading can be beneficial for handling irregular and non-compute-bound operations while keeping the system efficient.

Reconfigurable and heterogeneous hardware for LLMs. Reconfigurable hardware like FPGAs can be dynamically adjusted to best suit the latest LLM architecture and model compression methods. Additionally, it can be combined with heterogeneous hardware to explore new compute paradigms, such as adopting in-memory computing, to address memory-bound operations. This direction enables trade-offs between model quality and hardware efficiency.

Co-design for edge LLM applications. Edge LLM is challenging but we believe LLM-hardware co-design is the most promising approach for achieving this goal. Research in this direction will explore co-design solutions that overcome energy and resource constraints while ensuring uncompromising quality of LLM services.

4 LLM-TO-ACCELERATOR COMPILER

4.1 Background Study

High-Level Synthesis (HLS) [3][4] is vital for rapidly developing efficient, high-density hardware accelerators, enabling quick evaluation of different algorithmic choices. The challenge of enabling scalable compiler from LLM models to HLS-based accelerators lies in effectively exploring the vast design space, which can lead to sub-optimal solutions if not done well, undermining the productivity benefits of HLS. To tackle this challenge, in this section, we will introduce two compilation frameworks, ScaleHLS and HIDA, which can generate HLS accelerators directly from PyTorch models.

4.2 Proposed Works

4.2.1 ScaleHLS. ScaleHLS [11, 28–30] is a scalable HLS framework on multi-level Intermediate Representation (IR). ScaleHLS supports C/C++ and PyTorch as design entries. Once the inputs are parsed, ScaleHLS supports three levels of IR, *graph*, *loop*, and *directive*, to apply the HLS-oriented optimizations progressively.

On top of each level of IR, ScaleHLS provides a set of transform passes to optimize HLS designs. Meanwhile, we propose a Design Space Exploration (DSE) engine to automatically optimize the configurable design parameters and search for the Pareto-dominating design points in the latency-resource utilization space. Finally, the optimized IR is emitted as synthesizable HLS C/C++ code. Experimental results show that, comparing to the baseline designs without manual directives insertion and code-rewriting, that are only synthesized by Vitis HLS, ScaleHLS improves the performances with up to 3825.0× on representative neural network models.

4.2.2 HIDA. HIDA [31] is an HLS framework built upon ScaleHLS with hierarchical dataflow intermediate representations (IR) and optimizations, enabling the automated transformation of PyTorch or C/C++ program to efficient dataflow architectures. The core of HIDA is its novel dataflow IR, named *HIDA-IR*, which is designed for modeling dataflow at two distinct abstraction levels: *Functional* and *Structural*. This dual-level approach is critical for capturing the dataflow’s characteristics and its multi-level hierarchy, thereby facilitating effective optimizations. Another important aspect of HIDA is the introduction of *HIDA-OPT*, a new dataflow optimizer. This optimizer utilizes an intensity- and connection-aware dataflow parallelization algorithm, which can capture the computation complexity and interconnection topology of the dataflow nodes during the parallelization process. Despite being fully automated and able to handle various applications, HIDA achieved throughputs that were 8.54× and 1.29× higher than those of ScaleHLS and RTL-based neural network accelerators [34], respectively.

4.3 Future Directions

Spatial Architecture. Due to the substantial volume of parameters and intermediate computations involved in LLMs, the bottleneck in hardware acceleration frequently resides in the external memory bandwidth. Contrary to the Von Neumann architecture, which consistently battles the "memory wall," spatial architectures can leverage on-chip communication among tasks to minimize frequent external memory accesses. By overlapping the execution of distinct LLM layers and buffering only a subset of intermediate results on-chip, spatial architectures can markedly decrease on-chip memory requirements and overall latency. This approach presents a

compelling solution for LLM inference. Nonetheless, the automatic generation of spatial architectures remains challenging, opening vast avenues for innovation in compilation and architecture design.

Runtime Reconfiguration. To achieve spatial parallelization, tasks must be instantiated simultaneously on-chip. However, due to constrained computational and on-chip memory resources, it is infeasible to simultaneously map all layers of emerging LLMs on-chip, which significantly limits the scalability of spatial architectures. Consequently, runtime reconfiguration emerges as a crucial strategy for enabling scalable spatial solutions. The main challenge lies in automating the balance between spatial and sequential execution — that is, addressing the scheduling problem — to optimize the performance-energy trade-off.

Heterogeneous Computation. Accelerating LLMs presents a unique challenge due to their dual nature, being both computation-bound and memory-bound. The prompting phase of LLMs is dominated by General Matrix Multiply (GEMM) operators, making it computation-intensive. In contrast, the generation phase is dominated by General Matrix-Vector (GEMV) operations, demanding substantial memory bandwidth to keep the computation units engaged. This dual nature of LLMs unveils significant opportunities for heterogeneous computing solutions, where compilers assume an important role in the code generation for heterogeneous platforms and facilitating efficient communication between them.

5 LLM-AIDED DESIGN

5.1 Background Study

The existing related work regarding leveraging LLMs in the field of EDA could be divided into three categories [37]: (1) Assistant Chatbot for Enhanced Design Workflow: Assistant chatbots, such as ChipNemo [18], leverage LLMs to provide instant access to knowledge, streamlining the query-response process and reducing dependence on traditional search methods and wait times; (2) HDL and Script Generation: LLMs, such as those in AutoChip [22] and VeriGen [23], have shown their effectiveness in generating Verilog codes and EDA tool scripts from natural language instructions; (3) HDL Verification and Analysis: RTLFixer [24], enhances error detection and optimization in HDL codes through specialized prompting techniques. Other projects [13] focus on generating SystemVerilog Assertions (SVA) for security, highlighting LLMs’ role in improving HDL analysis and verification.

Despite the promising developments in LLM-aided design within EDA, several challenges remain: (1) Data Quality and Availability: The success of LLMs in EDA relies on access to high-quality, domain-specific data. However, the private nature of electronic designs and EDA tools limits this access, creating a data shortage that impedes LLM development for EDA; (2) Development of Specialized LLMs: There’s a vital need for LLMs designed for the complexities of electronic design. Generic models lack the precision for effective hardware code generation and EDA tool interaction.

5.2 Proposed Works

One use case of LLM-aided design is to harness LLMs for enhancing the debugging and verification processes for HLS code development. HLS, with its higher level of abstraction, can significantly improve design productivity as explained in Section 4.1.

Chrysalis Dataset. The cornerstone of our proposed work is the Chrysalis dataset, an extensive collection of HLS designs embedded with a diverse set of realistic bugs [25]. This dataset is meticulously curated from a wide range of open-source HLS benchmark suites, featuring over 1,500 designs, with both the version embedded with bugs and the corresponding bug-free version. This dataset serves not only as a tool for training our domain-specific LLM but also as a benchmark for evaluating the model’s proficiency in identifying and suggesting fixes for common and complex HLS bugs.

HLS-specific Debugging Assistant. Building upon the Chrysalis dataset, our next step involves the creation of an HLS-specific debugging assistant. This LLM will be fine-tuned to understand the intricacies of HLS code, enabling it to identify bugs, suggest corrections, and provide guidance directly within the developers’ workflow. By providing context-aware suggestions and corrections, the debugging assistant will significantly expedite the verification process, enhancing productivity and reducing the time-to-market for hardware designs.

5.3 Future Directions

LLM-Powered EDA Integration Suite. A promising direction for future research and development lies in crafting LLM-assisted design tools that integrate with existing EDA platforms. Such tools, leveraging specialized LLMs capable of understanding electronic design languages and processes, could revolutionize design optimization and error correction, streamlining the design cycle and enhancing the productivity of engineers. Through interactive learning experiences and tailored feedback, LLMs could significantly contribute to the development of a skilled workforce, ready to navigate the complexities of tomorrow’s electronic design challenges. These prospects not only underscore the potential of LLMs to reshape the EDA landscape but also highlight the need for future research in this fascinating new field.

6 CONCLUSION

In our study, we focused on optimizing LLMs to reduce inference latency and improve efficiency across various applications. We discussed methods to use multiple decoding heads for prediction, coupled with an optimized tree-based decoding strategy for parallel token processing to speed up the execution of LLMs.

We discussed LLM/hardware co-design to optimize hardware architectures for efficient execution, integrating model architecture exploration and multi-objective hardware optimization for improved system efficiency and accuracy. HLS frameworks like ScaleHLS and HIDA were explored for accelerating LLMs directly from PyTorch, envisioning automated generation of spatial architectures and heterogeneous computing solutions.

We also explore the advancements in the integration of LLMs into EDA and propose the establishment of the Chrysalis dataset and the crafting of an HLS-specific debugging assistant to streamline debugging and verification in HLS code development.

For each aspect mentioned above, we also outlined promising future directions for further research and exploration.

ACKNOWLEDGMENTS

This work is supported in part by the IBM-Illinois Discovery Accelerator Institute, AMD Center of Excellence at UIUC, AMD Heterogeneous Adaptive Compute Cluster (HACC) initiative, NSF 2117997

grant through the A3D3 institute, and Semiconductor Research Corporation (SRC) 2023-CT-3175 grant.

REFERENCES

- [1] Tianle Cai et al. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv:2401.10774*.
- [2] Charlie Chen et al. 2023. Accelerating large language model decoding with speculative sampling. *arXiv:2302.01318*.
- [3] Deming Chen et al. 2005. xPilot: A Platform-Based Behavioral Synthesis System. In *SRC Techcon*.
- [4] Jason Cong et al. 2022. FPGA HLS Today: Successes, Challenges, and Opportunities. In *TRETS*.
- [5] Tri Dao et al. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*.
- [6] Tim Dettmers et al. 2022. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*.
- [7] Yichao Fu et al. 2024. Break the sequential dependency of llm inference using lookahead decoding. *arXiv:2402.02057*.
- [8] Cong Hao et al. 2018. Deep neural network model and FPGA accelerator co-design: Opportunities and challenges. In *ICSICT*.
- [9] Cong Hao et al. 2019. FPGA/DNN co-design: An efficient design methodology for IoT intelligence on the edge. In *DAC*.
- [10] Seongmin Hong et al. 2022. DFX: A low-latency multi-FPGA appliance for accelerating transformer-based text generation. In *MICRO*.
- [11] Hyegang Jun et al. 2023. AutoScaleDSE: A scalable design space exploration engine for high-level synthesis. In *TRETS*.
- [12] Jordan Juravsky et al. 2024. Hydragen: High-Throughput LLM Inference with Shared Prefixes. *arXiv:2402.05099*.
- [13] Rahul Kande et al. 2023. Llm-assisted generation of hardware assertions. *arXiv:2306.14027*.
- [14] Achintya Kundu et al. 2024. Efficiently Distilling LLMs for Edge Applications. *arXiv preprint arXiv:2404.01353*.
- [15] Woosuk Kwon et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*.
- [16] Yaniv Leviathan et al. 2023. Fast inference from transformers via speculative decoding. In *ICML*.
- [17] Patrick Lewis et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*.
- [18] Mingjie Liu et al. 2023. Chipmemo: Domain-adapted llms for chip design. *arXiv:2311.00176*.
- [19] James M Ortega and Werner C Rheinboldt. 2000. Iterative solution of nonlinear equations in several variables. *Classics in Applied Mathematics*.
- [20] Andrea Santilli et al. 2023. Accelerating transformer inference for translation via parallel decoding. *arXiv:2305.10427*.
- [21] Mitchell Stern et al. 2018. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*.
- [22] Shailja Thakur et al. 2023. Autochip: Automating hdl generation using llm feedback. *arXiv:2311.04887*.
- [23] Shailja Thakur et al. 2023. Verigen: A large language model for verilog code generation. In *TRETS*.
- [24] YunDa Tsai et al. 2023. Rtlfixer: Automatically fixing rtl syntax errors with large language models. *arXiv:2311.16543*.
- [25] Lily Jiaxin Wan et al. 2024. Software/Hardware Co-design for LLM and Its Application for Design Verification. In *ASP-DAC*.
- [26] Jason Wei et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*.
- [27] Guangxuan Xiao et al. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *ICML*.
- [28] Hanchen Ye et al. 2022. ScaleHLS: A new scalable high-level synthesis framework on multi-level intermediate representation. In *HPCA*.
- [29] Hanchen Ye et al. 2022. ScaleHLS: a scalable high-level synthesis framework with multi-level transformations and optimizations. In *DAC*.
- [30] Hanchen Ye et al. 2023. High-level Synthesis for Domain Specific Computing. In *ISPD*.
- [31] Hanchen Ye et al. 2024. HIDA: A Hierarchical Dataflow Compiler for High-Level Synthesis. In *ASPLOS*.
- [32] Haoran You et al. 2023. Vitcod: Vision transformer acceleration via dedicated algorithm and accelerator co-design. In *HPCA*.
- [33] Shulin Zeng et al. 2024. FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGA. *arXiv:2401.03868*.
- [34] Xiaofan Zhang et al. 2018. DNNBuilder: An automated tool for building high-performance DNN hardware accelerators for FPGAs. In *ICCAD*.
- [35] Xiaofan Zhang et al. 2022. AutoDistill: An end-to-end framework to explore and distill hardware-efficient language models. *arXiv:2201.08539*.
- [36] Zhenyu Zhang et al. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv:2306.14048*.
- [37] Ruizhe Zhong et al. 2023. LLM4EDA: Emerging Progress in Large Language Models for Electronic Design Automation. *arXiv:2401.12224*.